Fine-tuning a Summarization Model

Introduction

In this chapter, we will explore how to use a Transformer model to convert long documents into short, concise summaries — a task known as **Text Summarization**.

This is one of the most challenging tasks in NLP because it requires:

- The ability to understand long passages of text
- The ability to retain key ideas and express them in coherent language

But when it works well, it significantly accelerates business workflows — experts no longer need to read entire lengthy documents.

One particularly exciting aspect of this chapter: we will build a **bilingual model** that can handle both **English and Spanish**.

By the end, our model will be able to summarize customer reviews like this:

> `"I loved The dash diet weight loss Solution. Never hungry. I would recommend this diet."` → Summary: `"Easy to follow!!!!"`

Step 1 — Building the Multilingual Corpus

Dataset Selection

We will use the **`mteb/amazon_reviews_multi`** dataset, which is a publicly available mirror of the original Multilingual Amazon Reviews Corpus. It contains Amazon product reviews in six languages. Each review comes with a short title — those titles will serve as our **target summaries**.

> **Why this dataset?** The original `amazon_reviews_multi` dataset was taken down by its data providers and is no longer accessible. The `mteb/amazon_reviews_multi` dataset is a publicly available version with the exact same structure and fields.

Let's load the English and Spanish datasets from Hugging Face Hub:

```
from datasets import load_dataset

spanish_dataset = load_dataset("mteb/amazon_reviews_multi", "es")
english_dataset = load_dataset("mteb/amazon_reviews_multi", "en")

english_dataset
```

Output:

```
DatasetDict({
    train: Dataset({
        features: ['review_id', 'product_id', 'reviewer_id', 'stars', 'review_body', 'review_title',
'language', 'product_category'],
        num_rows: 200000
    })
    validation: Dataset({
        features: ['review_id', 'product_id', 'reviewer_id', 'stars', 'review_body', 'review_title',
'language', 'product_category'],
        num_rows: 5000
    })
    test: Dataset({
        features: ['review_id', 'product_id', 'reviewer_id', 'stars', 'review_body', 'review_title',
'language', 'product_category'],
        num_rows: 5000
    })
})
```

As you can see, each language has 200,000 reviews in the train split, and 5,000 each in the validation and test splits. The fields we need are `review_body` (the full review) and `review_title` (our target summary).

Exploring the Data

Let's write a small function to look at a few random samples from the training set:

```
def show_samples(dataset, num_samples=3, seed=42):
    sample = dataset["train"].shuffle(seed=seed).select(range(num_samples))
    for example in sample:
        print(f"\n'>> Title: {example['review_title']}'")
        print(f"'>> Review: {example['review_body']}'")
```

```
show_samples(english_dataset)
```

Output:

```
'>> Title: Worked in front position, not rear'
'>> Review: 3 stars because these are not rear brakes as stated in the item description...'

'>> Title: meh'
'>> Review: Does it's job and it's gorgeous but mine is falling apart...'

'>> Title: Can't beat these for the money'
'>> Review: Bought this for handling miscellaneous aircraft parts...'
```

> **Try it out!** Change the `seed` value in `Dataset.shuffle()` to see different reviews. If you speak Spanish, try calling `show_samples(spanish_dataset)` — do the titles feel like genuine summaries?

Analyzing Product Categories

Training on 400,000 reviews would take a very long time on a single GPU, so we'll focus on a specific product category. Let's convert the dataset to a pandas DataFrame to see what categories are available:

```
english_dataset.set_format("pandas")
english_df = english_dataset["train"][:]

# Show counts for top 20 product categories
english_df["product_category"].value_counts()[:20]
```

Output:

```
home            17679
apparel         15951
wireless        15717
other           13418
beauty          12091
drugstore       11730
kitchen         10382
toy              8745
sports           8277
```

```
automotive                7506
lawn_and_garden           7327
home_improvement          7136
pet_products              7082
digital_ebook_purchase    6749
pc                        6401
electronics               6186
office_product            5521
shoes                     5197
grocery                   4730
book                      3756
```

Amazon started as a bookstore, so let's focus on the `book` and `digital_ebook_purchase` categories!

Filtering for Books

```
def filter_books(example):
    return (
        example["product_category"] == "book"
        or example["product_category"] == "digital_ebook_purchase"
    )
```

Before applying the filter, let's reset the dataset format back to `"arrow"`:

```
english_dataset.reset_format()
```

Now apply the filter and preview some samples:

```
spanish_books = spanish_dataset.filter(filter_books)
english_books = english_dataset.filter(filter_books)

show_samples(english_books)
```

Output:

```
'>> Title: I'm dissapointed.'
'>> Review: I guess I had higher expectations for this book from the reviews...'

'>> Title: Good art, good price, poor design'
```

'>> Review: I had gotten the DC Vintage calendar the past two years...'

'>> Title: Helpful'
'>> Review: Nearly all the tips useful and. I consider myself an intermediate to advanced user of OneNote...'

Notice that the reviews aren't exclusively about books — there are calendars and digital tools like OneNote. But the domain is still good enough to train a summarization model.

Building the Bilingual Dataset

Now let's combine the English and Spanish reviews into a single `DatasetDict`. We'll use the `concatenate_datasets()` function from the `datasets` library to join them side by side:

```
from datasets import concatenate_datasets, DatasetDict

books_dataset = DatasetDict()

for split in english_books.keys():
    books_dataset[split] = concatenate_datasets(
        [english_books[split], spanish_books[split]]
    )
    books_dataset[split] = books_dataset[split].shuffle(seed=42)

# Peek at a few examples
show_samples(books_dataset)
```

Output:

'>> Title: Easy to follow!!!!'
'>> Review: I loved The dash diet weight loss Solution. Never hungry. I would recommend this diet...'

'>> Title: PARCIALMENTE DAÑADO'
'>> Review: Me llegó el día que tocaba, junto a otros libros que pedí...'

'>> Title: no lo he podido descargar'
'>> Review: igual que el anterior'

We can see a mix of English and Spanish reviews.

Checking Word Distribution & Filtering Short Titles

Before preparing the training corpus, it's important to look at the distribution of words in the reviews and titles. This is especially critical in summarization tasks, because if the reference summaries are very short (e.g., 1–2 words), the model will learn to produce only 1–2 word outputs.

To avoid this, let's filter out very short titles so the model learns to generate more informative summaries:

books_dataset = books_dataset.filter(lambda x: len(x["review_title"].split()) > 2)

Step 2 — Models for Text Summarization

Think of Summarization Like Translation

Text summarization is similar to machine translation — we are "translating" a review into its condensed version. This is why most Transformer summarization models use an **encoder-decoder architecture**.

Here is a summary of popular pretrained models:

Model	Description	Multilingual?
GPT-2	Auto-regressive language model. Can generate summaries if you append `"TL;DR"` to the input.	❌
PEGASUS	Uses masked sentence prediction as pretraining objective. Scores highly on popular summarization benchmarks.	❌
T5	Completes all NLP tasks in a text-to-text framework. Input format for summarization: `summarize: ARTICLE`.	❌
mT5	Multilingual version of T5. Pretrained on the multilingual Common Crawl corpus (mC4) in 101 languages.	✅
BART	Has both encoder and decoder. Combines BERT and GPT-2 pretraining by learning to reconstruct corrupted input.	❌
mBART-50	Multilingual version of BART, pretrained on 50 languages.	✅

Why mT5?

We'll use **mT5**. In T5, every NLP task is handled with a prompt prefix (e.g., `summarize:`). mT5 doesn't use prefixes, but it is just as versatile as T5 and has the added benefit of being multilingual.

> **Try it out!** After finishing this chapter, compare mT5 vs mBART. As a bonus, try fine-tuning T5 on English-only reviews — just remember to prepend `summarize:` to the input during preprocessing.

Step 3 — Data Preprocessing

Loading the Tokenizer

We'll use the `mt5-small` checkpoint so we can fine-tune the model in a reasonable amount of time:

```
from transformers import AutoTokenizer

model_checkpoint = "google/mt5-small"
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
```

> **Tip:** When starting an NLP project, always begin with a "small" model on a subset of your data. This allows you to debug and iterate quickly. Once you're satisfied with the results, simply swap the model checkpoint to scale up!

Let's test the tokenizer on a small example:

```
inputs = tokenizer("I loved reading the Hunger Games!")
inputs
```

Output:

```
{'input_ids': [336, 259, 28387, 11807, 287, 62893, 295, 12507, 1],
 'attention_mask': [1, 1, 1, 1, 1, 1, 1, 1, 1]}
```

Let's decode the input IDs to understand what tokenizer is being used:

```
tokenizer.convert_ids_to_tokens(inputs.input_ids)
```

Output:

['_I', '_', 'loved', '_reading', '_the', '_Hung', 'er', '_Games', '</s>']

The special Unicode character _ and the end-of-sequence token `</s>` tell us this is a **SentencePiece tokenizer**, which uses the **Unigram segmentation algorithm**. Unigram is particularly well-suited for multilingual corpora because it handles accents, punctuation, and whitespace-free languages (like Japanese) in an agnostic way.

The Preprocess Function

There is a subtle point in summarization: our **labels are also text**, so they could exceed the model's maximum context size. Therefore, we must apply truncation to both the reviews and the titles:

```
max_input_length = 512
max_target_length = 30

def preprocess_function(examples):
    model_inputs = tokenizer(
        examples["review_body"],
        max_length=max_input_length,
        truncation=True,
    )
    labels = tokenizer(
        examples["review_title"],
        max_length=max_target_length,
        truncation=True
    )
    model_inputs["labels"] = labels["input_ids"]
    return model_inputs
```

Here, `max_input_length = 512` is set for reviews and `max_target_length = 30` for titles — because reviews are typically much longer than titles.

Now let's tokenize the entire corpus using `Dataset.map()`:

```
tokenized_datasets = books_dataset.map(preprocess_function, batched=True)
```

> **Tip:** Using `batched=True` processes data in batches of 1,000 examples by default, and takes advantage of multithreading in Transformers' fast tokenizers.

Step 4 — ROUGE Score: The Summarization Metric

Why ROUGE?

Measuring text generation is not as straightforward as classification. A single review can have many valid summaries, and doing an exact match between generated and reference summaries is not meaningful.

The most widely used metric for summarization is the **ROUGE score** (Recall-Oriented Understudy for Gisting Evaluation). The core idea is to compare a generated summary against a human-written reference summary.

Computing Precision and Recall

Suppose we have:

generated_summary = "I absolutely loved reading the Hunger Games"
reference_summary = "I loved reading the Hunger Games"

Recall measures how much of the reference summary is captured in the generated summary:

Recall = (overlapping words) / (total words in reference)
= 6 / 6 = 1.0 ← perfect recall

But what if our generated summary was `"I really really loved reading the Hunger Games all night"`? Recall would still be perfect, but the summary is verbose. This is why we also need **Precision**:

Precision = (overlapping words) / (total words in generated summary)

Verbose summary: 6/10 = 0.6 ← much worse
Concise summary: 6/7 = 0.86 ← much better

Installing and Using ROUGE

!pip install rouge_score

```
import evaluate
rouge_score = evaluate.load("rouge")

scores = rouge_score.compute(
    predictions=[generated_summary],
    references=[reference_summary]
)
scores
```

Output:

```
{'rouge1': AggregateScore(low=Score(precision=0.86, recall=1.0, fmeasure=0.92), ...),
 'rouge2': AggregateScore(low=Score(precision=0.67, recall=0.8, fmeasure=0.73), ...),
 'rougeL': AggregateScore(low=Score(precision=0.86, recall=1.0, fmeasure=0.92), ...),
 'rougeLsum': AggregateScore(...)}
```

There's a lot of information here. Let's break it down:

The `datasets` library computes confidence intervals for precision, recall, and F1-score — shown as `low`, `mid`, and `high`.

- **rouge1** — overlap of individual words (unigrams)
- **rouge2** — overlap of word pairs (bigrams)
- **rougeL** — longest matching sequence of words (longest common substring)
- **rougeLsum** — rougeL computed over the full summary

Let's inspect the `mid` score:

```
scores["rouge1"].mid
```

Output:

```
Score(precision=0.86, recall=1.0, fmeasure=0.92)
```

The numbers match our manual calculation perfectly.

Step 5 — Building a Strong Baseline

The Lead-3 Baseline

A common baseline for text summarization is to take the **first three sentences** of an article — this is called the **lead-3 baseline**.

If we naively split on full stops, abbreviations like "U.S." or "U.N." would cause problems. So we'll use the `nltk` library instead:

```
!pip install nltk
import nltk
nltk.download("punkt")

from nltk.tokenize import sent_tokenize

def three_sentence_summary(text):
    return "\n".join(sent_tokenize(text)[:3])

print(three_sentence_summary(books_dataset["train"][1]["review_body"]))
```

Output:

```
'I grew up reading Koontz, and years ago, I stopped, convinced i had "outgrown" him.'
'Still, when a friend was looking for something suspenseful to read, I suggested Koontz.'
'She found Strangers.'
```

Now let's compute ROUGE scores on the validation set:

```
def evaluate_baseline(dataset, metric):
    summaries = [three_sentence_summary(text) for text in dataset["review_body"]]
    return metric.compute(predictions=summaries, references=dataset["review_title"])

import pandas as pd

score = evaluate_baseline(books_dataset["validation"], rouge_score)
rouge_names = ["rouge1", "rouge2", "rougeL", "rougeLsum"]
rouge_dict = dict((rn, round(score[rn].mid.fmeasure * 100, 2)) for rn in rouge_names)
rouge_dict
```

Output:

```
{'rouge1': 16.74, 'rouge2': 8.83, 'rougeL': 15.6, 'rougeLsum': 15.96}
```

The `rouge2` score is quite low — because review titles are typically very concise, while the lead-3 baseline is far more verbose.

Step 6 — Fine-Tuning mT5 with the Trainer API

Loading the Model

```
from transformers import AutoModelForSeq2SeqLM

model = AutoModelForSeq2SeqLM.from_pretrained(model_checkpoint)
```

For sequence-to-sequence tasks, we keep all of the network's weights. Unlike text classification where we had to swap out the head, there's no need for that here.

Logging in to Hugging Face Hub

```
from huggingface_hub import notebook_login

notebook_login()
```

Or in a terminal:

```
huggingface-cli login
```

Setting Training Arguments

```
from transformers import Seq2SeqTrainingArguments

batch_size = 8
num_train_epochs = 8
logging_steps = len(tokenized_datasets["train"]) // batch_size
model_name = model_checkpoint.split("/")[-1]

args = Seq2SeqTrainingArguments(
    output_dir=f"{model_name}-finetuned-amazon-en-es",
    evaluation_strategy="epoch",
    learning_rate=5.6e-5,
    per_device_train_batch_size=batch_size,
    per_device_eval_batch_size=batch_size,
    weight_decay=0.01,
    save_total_limit=3,
```

```
    num_train_epochs=num_train_epochs,
    predict_with_generate=True,
    logging_steps=logging_steps,
    push_to_hub=True,
)
```

Key points:

- `predict_with_generate=True` makes the model generate actual summaries during evaluation (instead of just computing loss) so we can calculate real ROUGE scores each epoch.
- `save_total_limit=3` keeps only the 3 most recent checkpoints — even the "small" version of mT5 takes about 1 GB of disk space.
- `push_to_hub=True` automatically uploads the model to the Hub when training is done.

The compute_metrics Function

We need a `compute_metrics()` function to calculate ROUGE scores during training. This involves decoding the predicted token IDs and reference labels back to text:

```
import numpy as np

def compute_metrics(eval_pred):
    predictions, labels = eval_pred

    # Decode generated summaries into text
    decoded_preds = tokenizer.batch_decode(predictions, skip_special_tokens=True)

    # Replace -100 in the labels (we can't decode them)
    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)

    # Decode reference summaries into text
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)

    # ROUGE expects a newline after each sentence
    decoded_preds = ["\n".join(sent_tokenize(pred.strip())) for pred in decoded_preds]
    decoded_labels = ["\n".join(sent_tokenize(label.strip())) for label in decoded_labels]

    # Compute ROUGE scores
    result = rouge_score.compute(
        predictions=decoded_preds,
```

```
        references=decoded_labels,
        use_stemmer=True
    )

    # Extract the median scores
    result = {key: value.mid.fmeasure * 100 for key, value in result.items()}
    return {k: round(v, 4) for k, v in result.items()}
```

The Data Collator

mT5 is an encoder-decoder Transformer, so there's a subtle detail when preparing batches: during decoding, the **labels must be shifted one step to the right**. This ensures the decoder only sees previous ground-truth labels, not the current or future ones.

Transformers' `DataCollatorForSeq2Seq` handles this automatically:

```
from transformers import DataCollatorForSeq2Seq

data_collator = DataCollatorForSeq2Seq(tokenizer, model=model)
```

We also need to remove the string columns, since the collator can't pad them:

```
tokenized_datasets = tokenized_datasets.remove_columns(
    books_dataset["train"].column_names
)
```

Let's test the collator on a small batch:

```
features = [tokenized_datasets["train"][i] for i in range(2)]
data_collator(features)
```

Output:

```
{'attention_mask': tensor([[1, 1, ..., 0, 0],
                           [1, 1, ..., 1, 1]]),
 'input_ids': tensor([[1494, 259, ...],
                      [259, 27531, ...]]),
 'labels': tensor([[7483, 259, 2364, 15695, 1, -100],
                   [259, 27531, 13483, 259, 7505, 1]]),
 'decoder_input_ids': tensor([[0, 7483, 259, 2364, 15695, 1],
```

[0, 259, 27531, 13483, 259, 7505]])}

Things to notice:

- The first example is shorter than the second, so `[PAD]` tokens (ID 0) are appended to its `input_ids` and `attention_mask`.
- The `labels` are padded with `-100` so the loss function ignores them.
- The `decoder_input_ids` have the labels shifted one step to the right, with `[PAD]` at the start.

Creating the Trainer and Starting Training

```
from transformers import Seq2SeqTrainer

trainer = Seq2SeqTrainer(
    model,
    args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
    data_collator=data_collator,
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,
)

trainer.train()
```

During training, you will see the training loss decrease and the ROUGE scores increase with each epoch.

Final Evaluation

```
trainer.evaluate()
```

Output:

```
{'eval_loss': 3.028524398803711,
 'eval_rouge1': 16.9728,
 'eval_rouge2': 8.2969,
 'eval_rougeL': 16.8366,
 'eval_rougeLsum': 16.851,
 'eval_gen_len': 10.1597,
```

'eval_runtime': 6.1054,
'eval_samples_per_second': 38.982,
'eval_steps_per_second': 4.914}

Our fine-tuned model clearly beats the lead-3 baseline!

Pushing to the Hub

trainer.push_to_hub(commit_message="Training complete", tags="summarization")

Output:

'https://huggingface.co/huggingface-course/mt5-finetuned-amazon-en-es/commit/aa0536b...'

The `tags="summarization"` argument ensures the Hub widget shows up as a summarization pipeline.

Step 7 — Fine-Tuning mT5 with Accelerate

Setting Everything Up

Fine-tuning with Accelerate is very similar to the text classification example from Chapter 3. The key difference is that during training, we need to explicitly **generate summaries** and **compute ROUGE scores**.

First, set the dataset format to `"torch"`:

tokenized_datasets.set_format("torch")

Re-load the model fresh:

model = AutoModelForSeq2SeqLM.from_pretrained(model_checkpoint)

Create the data collator and DataLoaders:

from torch.utils.data import DataLoader

batch_size = 8

```
train_dataloader = DataLoader(
    tokenized_datasets["train"],
    shuffle=True,
    collate_fn=data_collator,
    batch_size=batch_size,
)

eval_dataloader = DataLoader(
    tokenized_datasets["validation"],
    collate_fn=data_collator,
    batch_size=batch_size,
)
```

Set up the optimizer:

```
from torch.optim import AdamW

optimizer = AdamW(model.parameters(), lr=2e-5)
```

Prepare everything with Accelerator:

```
from accelerate import Accelerator

accelerator = Accelerator()
model, optimizer, train_dataloader, eval_dataloader = accelerator.prepare(
    model, optimizer, train_dataloader, eval_dataloader
)
```

> If you're training on a TPU, all of the above code must be placed inside a dedicated training function.

Learning Rate Schedule & Post-processing

Set up a linear learning rate scheduler:

```
from transformers import get_scheduler

num_train_epochs = 10
num_update_steps_per_epoch = len(train_dataloader)
num_training_steps = num_train_epochs * num_update_steps_per_epoch
```

```
lr_scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=0,
    num_training_steps=num_training_steps,
)
```

For the ROUGE metric, we need to split generated summaries into sentences separated by newlines:

```
def postprocess_text(preds, labels):
    preds = [pred.strip() for pred in preds]
    labels = [label.strip() for label in labels]

    # ROUGE expects a newline after each sentence
    preds = ["\n".join(nltk.sent_tokenize(pred)) for pred in preds]
    labels = ["\n".join(nltk.sent_tokenize(label)) for label in labels]

    return preds, labels
```

Creating a Hub Repository

```
from huggingface_hub import get_full_repo_name

model_name = "mt5-finetuned-amazon-en-es-accelerate"
repo_name = get_full_repo_name(model_name)
repo_name
```

Output:

```
'lewtun/mt5-finetuned-amazon-en-es-accelerate'
```

```
from huggingface_hub import Repository

output_dir = "results-mt5-finetuned-amazon-en-es-accelerate"
repo = Repository(output_dir, clone_from=repo_name)
```

The Training Loop

The training loop has four main steps per epoch:

1. Train the model on all examples in `train_dataloader`
2. At the end of each epoch, generate tokens and decode them into summaries
3. Compute ROUGE scores
4. Save the checkpoint and push it to the Hub asynchronously (using `blocking=False` so training isn't halted)

```
from tqdm.auto import tqdm
import torch
import numpy as np

progress_bar = tqdm(range(num_training_steps))

for epoch in range(num_train_epochs):
    # ── Training ──
    model.train()
    for step, batch in enumerate(train_dataloader):
        outputs = model(**batch)
        loss = outputs.loss
        accelerator.backward(loss)

        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()
        progress_bar.update(1)

    # ── Evaluation ──
    model.eval()
    for step, batch in enumerate(eval_dataloader):
        with torch.no_grad():
            generated_tokens = accelerator.unwrap_model(model).generate(
                batch["input_ids"],
                attention_mask=batch["attention_mask"],
            )
            generated_tokens = accelerator.pad_across_processes(
                generated_tokens, dim=1, pad_index=tokenizer.pad_token_id
            )
            labels = accelerator.pad_across_processes(
                batch["labels"], dim=1, pad_index=tokenizer.pad_token_id
            )

            generated_tokens = accelerator.gather(generated_tokens).cpu().numpy()
            labels = accelerator.gather(labels).cpu().numpy()
```

```
        # Replace -100 in labels since we can't decode them
        labels = np.where(labels != -100, labels, tokenizer.pad_token_id)
        if isinstance(generated_tokens, tuple):
            generated_tokens = generated_tokens[0]

        decoded_preds = tokenizer.batch_decode(
            generated_tokens, skip_special_tokens=True
        )
        decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)
        decoded_preds, decoded_labels = postprocess_text(decoded_preds, decoded_labels)

        rouge_score.add_batch(predictions=decoded_preds, references=decoded_labels)

    # ── Compute Metrics ──
    result = rouge_score.compute()
    result = {key: value.mid.fmeasure * 100 for key, value in result.items()}
    result = {k: round(v, 4) for k, v in result.items()}
    print(f"Epoch {epoch}:", result)

    # ── Save & Upload ──
    accelerator.wait_for_everyone()
    unwrapped_model = accelerator.unwrap_model(model)
    unwrapped_model.save_pretrained(output_dir, save_function=accelerator.save)
    if accelerator.is_main_process:
        tokenizer.save_pretrained(output_dir)
        repo.push_to_hub(
            commit_message=f"Training in progress epoch {epoch}", blocking=False
        )
```

Output:

```
Epoch 0: {'rouge1': 5.6351, 'rouge2': 1.1625, 'rougeL': 5.4866, 'rougeLsum': 5.5005}
Epoch 1: {'rouge1': 9.8646, 'rouge2': 3.4106, 'rougeL': 9.9439, 'rougeLsum': 9.9306}
Epoch 2: {'rouge1': 11.0872, 'rouge2': 3.3273, 'rougeL': 11.0508, 'rougeLsum': 10.9468}
Epoch 3: {'rouge1': 11.8587, 'rouge2': 4.8167, 'rougeL': 11.7986, 'rougeLsum': 11.7518}
Epoch 4: {'rouge1': 12.9842, 'rouge2': 5.5887, 'rougeL': 12.7546, 'rougeLsum': 12.7029}
Epoch 5: {'rouge1': 13.4628, 'rouge2': 6.4598, 'rougeL': 13.312, 'rougeLsum': 13.2913}
Epoch 6: {'rouge1': 12.9131, 'rouge2': 5.8914, 'rougeL': 12.6896, 'rougeLsum': 12.5701}
Epoch 7: {'rouge1': 13.3079, 'rouge2': 6.2994, 'rougeL': 13.1536, 'rougeLsum': 13.1194}
Epoch 8: {'rouge1': 13.96,   'rouge2': 6.5998, 'rougeL': 13.9123, 'rougeLsum': 13.7744}
Epoch 9: {'rouge1': 14.1192, 'rouge2': 7.0059, 'rougeL': 14.1172, 'rougeLsum': 13.9509}
```

The results are nearly identical to what we got using the Trainer API.

Step 8 — Using the Fine-Tuned Model

After pushing to the Hub, you can use the model via the inference widget or a `pipeline` object:

from transformers import pipeline

hub_model_id = "huggingface-course/mt5-small-finetuned-amazon-en-es"
summarizer = pipeline("summarization", model=hub_model_id)

A function to display model output side-by-side with the original review and human title:

```
def print_summary(idx):
    review = books_dataset["test"][idx]["review_body"]
    title = books_dataset["test"][idx]["review_title"]
    summary = summarizer(books_dataset["test"][idx]["review_body"])[0]["summary_text"]

    print(f"'>>> Review: {review}'")
    print(f"\n'>>> Title: {title}'")
    print(f"\n'>>> Summary: {summary}'")
```

English Review Example

print_summary(100)

Output:

'>>> Review: Nothing special at all about this product... the book is too small and stiff and hard to write in...'
'>>> Title: Not impressed at all... buy something else'
'>>> Summary: Nothing special at all about this product'

The model performs abstractive summarization by pulling key phrases from the review.

Spanish Review Example

print_summary(0)

Output:

'>>> Review: Es una trilogia que se hace muy facil de leer. Me ha gustado, no me esperaba el final para nada'
'>>> Title: Buena literatura para adolescentes'
'>>> Summary: Muy facil de leer'

The summary translates to: *"Very easy to read"* — taken directly from the review. This is a great demonstration of mT5's multilingual capability!

What Did We Learn?

Topic	Key Takeaway
Dataset	Used `mteb/amazon_reviews_multi` — English & Spanish book reviews (public replacement for the defunct `amazon_reviews_multi`)
Model	mT5-small — multilingual, encoder-decoder architecture
Tokenizer	SentencePiece (Unigram) — ideal for multilingual corpora
Metric	ROUGE score — combines recall, precision, and F1
Baseline	Lead-3 — simply takes the first 3 sentences
Fine-tuning	`Seq2SeqTrainer` or Accelerate — both equally effective

Thank you for going through this notebook. If you found it helpful, feel free to share feedback or suggestions. Full code and resources are available on GitHub and Kaggle.